



DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Learn

What is DBSCAN?

DBSCAN is a popular unsupervised clustering algorithm that groups together data points that are closely packed (dense regions) and marks points in low-density regions as outliers or noise.

Full form: **Density-Based Spatial Clustering of Applications with Noise**

Why is DBSCAN useful?

- Can detect clusters of arbitrary shape.
 - Handles noise and outliers naturally.
 - Does not require you to specify the number of clusters in advance (unlike K-Means).
-

How does it work?

DBSCAN uses two main parameters:

- **eps**: Maximum distance between two points to be considered neighbors.
- **min_samples**: Minimum number of points required to form a dense region (cluster).

Steps:

1. Choose an unvisited point.
2. If it has at least **min_samples** points within **eps**, form a cluster.
3. Expand the cluster by recursively checking neighbors.
4. Mark points that don't belong to any cluster as noise.



Key Terms

- **Core Point:** Has at least **min_samples** points within **eps**.
 - **Border Point:** Within **eps** of a core point but not a core point itself.
 - **Noise Point:** Not a core or border point.
-

Practice

Let's implement DBSCAN in Python using scikit-learn.

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Generate data
X, _ = make_moons(n_samples=300, noise=0.1)

# Apply DBSCAN
model = DBSCAN(eps=0.3, min_samples=5)
labels = model.fit_predict(X)

# Visualize results
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("DBSCAN Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Know More

Frequently Asked Questions

Q1. How is DBSCAN different from K-Means?

- DBSCAN can find **non-spherical clusters** and **noise**, while K-Means assumes **spherical clusters** and needs **k** in advance.

Q2. How do I choose **eps** and **min_samples**?

- Try using a **k-distance graph** to select **eps**.
- A rule of thumb is **min_samples = 2 * number of features**.

Q3. Can DBSCAN handle high-dimensional data?

It struggles in high dimensions due to the **curse of dimensionality**.

Q4. What happens if **eps** is too small or large?

- Too small: Many small clusters or only noise.
- Too large: Clusters merge or only one big cluster.

Q5. Is DBSCAN deterministic?

Yes, the same input always gives the same result.

DBSCAN is perfect for clustering complex shapes and dealing with noisy datasets. Try it when K-Means doesn't work well.